EE 419 Project 1 Design Document

Geeoon Chung and Nate Snyder

Our decoder generally followed the design laid out by the specifications.

For our level 1, decoded the length of the message by following majority rules for the 3-bit repetition of the bits. After that, we deinterleaved the bits by indexing every 3 bits and placing every bit in that triplet to the correct output position. Afterwards, we converted every 8 bits of the resulting array into a value, then interpreted that 8 bit value as ASCII.

For our level 2, we implemented the Viterbi soft decoder. This was split up into stages. For each stage, where a stage corresponds to t = 1, 2, …, n. We precomputed the expected output for every state and input combination in order to speed up our algorithm. Then for each pair of inputs (i.e., the 4-QAM symbol), we computed the weight for each transition. The transition was then summed with the weight of the current state to determine the new shortest paths for the new stages. We also keep track of the shortest path taken for each new stage. This way, we are able to compute the shortest path as we travel from t = 1 to the last input. And because we're keeping track of the shortest path for each state, we do not need to do any backtracking to find the shortest path. Since we know the input is padded with two zeros at the end, we just select state 0 and look at the shortest path from that state. This way, we have an efficient algorithm that runs in O(n) given the fixed G matrix, where n is the length of the message.

For our level 3, we broke the signal up into chunks of nfft (64) symbols, then performed the FFT of each chunk. This was a direct inverse of the level 3 in the transmitter.

For our level 4, we used correlation to find the preamble. Using the preamble, we computed the 4-QAM symbols using a simple function, then the IFFT of the symbols. That kernel was correlated across the entire received signal to create our convolution. By finding the first index in the convolution that met a "similarity" threshold, we were able to reliably find the index of the start of the preamble, regardless of noise, padding, or the message coincidentally containing part of the preamble. Using extensive testing, we were able to find a threshold constant of 1 that worked well for all SNR. After finding the preamble, we decoded the length using techniques from level 3 and 1 to determine the actual length of our entire packet, then trimmed off the leading zeros and trailing zeros from our input.

To go into more depth, we inferred the length of our packet by getting the nfft (64) symbols after the end of the preamble, then performing an FFT on that chunk. We used the same majority rules technique to decode the length.

We did not implement any extra capabilities in our receiver.